

本小节内容

自增自减运算符

自增自减运算符与取值运算符

自增自减运算符

自增、自减运算符和其他运算符有很大的区别，**因为其他运算符除赋值运算符可以改变变量本身的值外，不会有这种效果。**自增、自减就是对变量自身进行加 1、减 1 操作，那么有了加法和减法运算符为什么还要发明这种运算符呢？原因是自增和自减来源于 B 语言，当时 Ken Thompson 和 Dennis M. Ritchie (C 语言的发明者) 为了不改变程序员的编写习惯，在 C 语言中保留了 B 语言中的自增和自减。因为自增、自减会改变变量的值，所以自增和自减不能用于常量！

下例中的 `j=i++>-1`，对于后++或者后--，首先我们需要去掉++或--运算符，也就是首先计算 `j=i>-1`，因为 i 本身等于 -1，所以得到 j 的值为 0，接着单独计算 `i++`，也就是对 i 加 1，所以 i 从 -1 加 1 得到 0，因此 `printf("i=%d,j=%d\n",i,j);` 语句的执行结果是 0 和 0。请看下例：

```
#include <stdio.h>
```

```
int main() {
```

```
    int i=-1,j;
```

```
    //5++;//如果打开该句，会造成编译不通
```

```
    j=i++>-1;//后++等价于 j=i>-1; i=i+1;
```

```
    printf ("i=%d,j=%d \n",i,j);
```

```
    return 0;
```

```
}
```

对于前加加，前减减，也就是++i 这种，直接按优先级进行运算即可，不再详述。

自增自减运算符与取值运算符（408 初试不需要掌握，初试考 C 的学校需要，部分机试答案含有）

虽然上面已经对自增自减运算符进行了讲解，但是当自增自减运算符与取值运算符结合使用时，会有一定的难度，原理与上面是一致的。我们再次来练习一下

```
#include <stdio.h>
#include <stdlib.h>

//只有比后增优先级高的操作符，才会作为一个整体，如()、[]
int main()
{
    int a[3]={2,7,8};
    int *p;
    int j;
    p=a;
    j=*p++; //先把*p 的值赋给 j，然后对 p 加 1
    printf("a[0]=%d,j=%d,*p=%d\n",a[0],j,*p);
    j=p[0]++; //先把 p[0]赋给 j，然后对 p[0]加 1
    printf("a[0]=%d,j=%d,*p=%d\n",a[0],j,*p);
    return 0;
}
```